

Data Visualization Using Python

Name: Simhadri Itrajula

Internship: Shadow Fox Data Science

Task : Beginner Level

Introduction

Visualization helps understand data using graphs.

Matplotlib

Matplotlib provides detailed control over plots.

- Line chart
- Bar chart
- Histogram
- Scatter plot
- Pie chart

Seaborn

Seaborn offers simpler syntax with more attractive statistical graphs.

- Line plot
- Bar plot
- Histogram plot
- Scatter plot
- Box plot
- Count plot

Comparison

Matplotlib and Seaborn are Python visualization libraries , where Matplotlib provides detailed control over plots, while Seaborn offers simpler syntax with more attractive statistical graphics.

Conclusion

Matplotlib and Seaborn help visualize data effectively.

INTRODUCTION

Data visualization is the process of representing data in graphical or pictorial form such as charts, graphs, and plots. It plays a crucial role in data science because it helps transform raw data into meaningful insights that are easy to understand.

In real-world scenarios, large datasets can be difficult to interpret when presented in tabular form. Visualization simplifies this process by highlighting trends, patterns, correlations, and outliers in the data. For example, instead of analyzing thousands of rows of sales data, a simple line chart can quickly show whether sales are increasing or decreasing over time.

Visualization is widely used in fields such as business analytics, healthcare, finance, and research. It helps decision-makers make informed choices based on data-driven insights.

Python provides powerful libraries for data visualization, among which **Matplotlib** and **Seaborn** are the most commonly used. These libraries allow users to create a wide variety of graphs and customize them according to their needs.

MATPLOTLIB

Matplotlib is one of the most widely used libraries in Python for data visualization. It was introduced by John Hunter and is built on top of the NumPy library. It provides a flexible and powerful way to create different types of visualizations.

Matplotlib is considered a low-level library because it gives users complete control over every element of a graph, including colors, labels, axes, and styles. It is especially useful when creating customized plots or publication-quality figures.

Some of the plots in matplotlib:

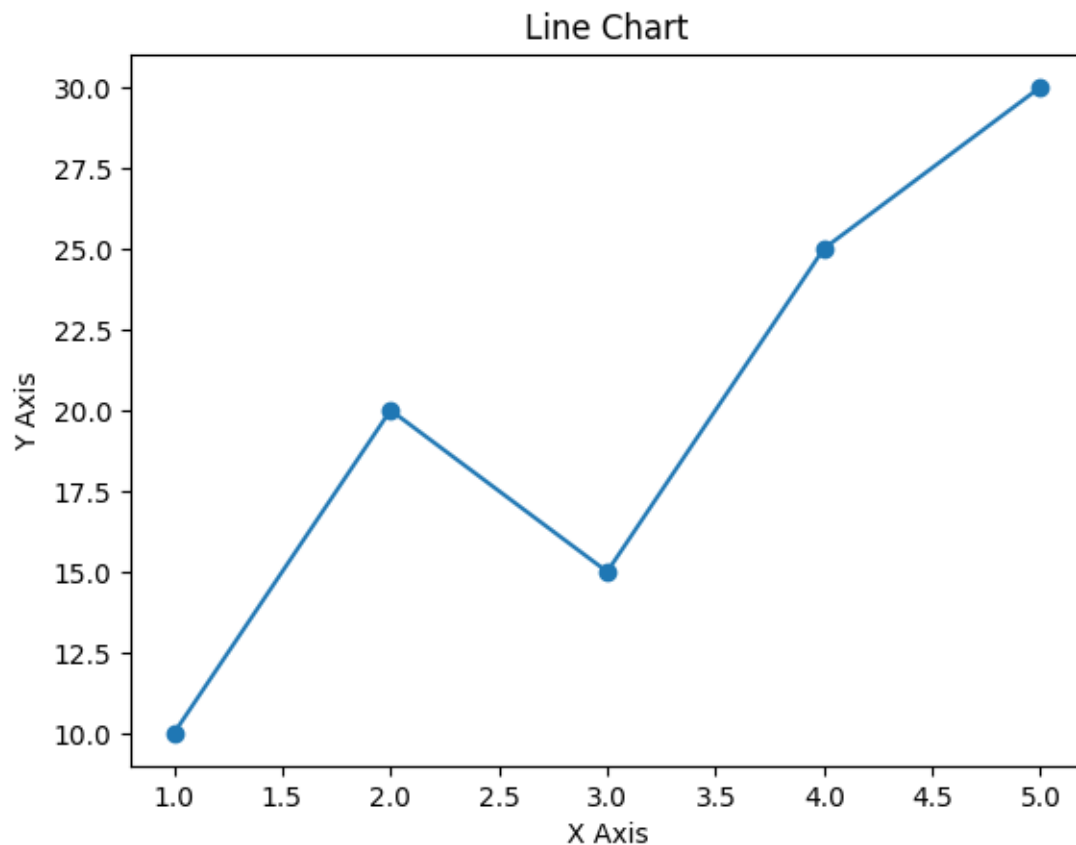
LINE CHART

A line chart is one of the most basic and commonly used types of plots. It connects individual data points with straight lines to show trends over time or continuous data.

```
▶ #LINE CHART
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [10,20,15,25,30]

plt.plot(x, y, marker='o')
plt.title("Line Chart")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```



Explanation:

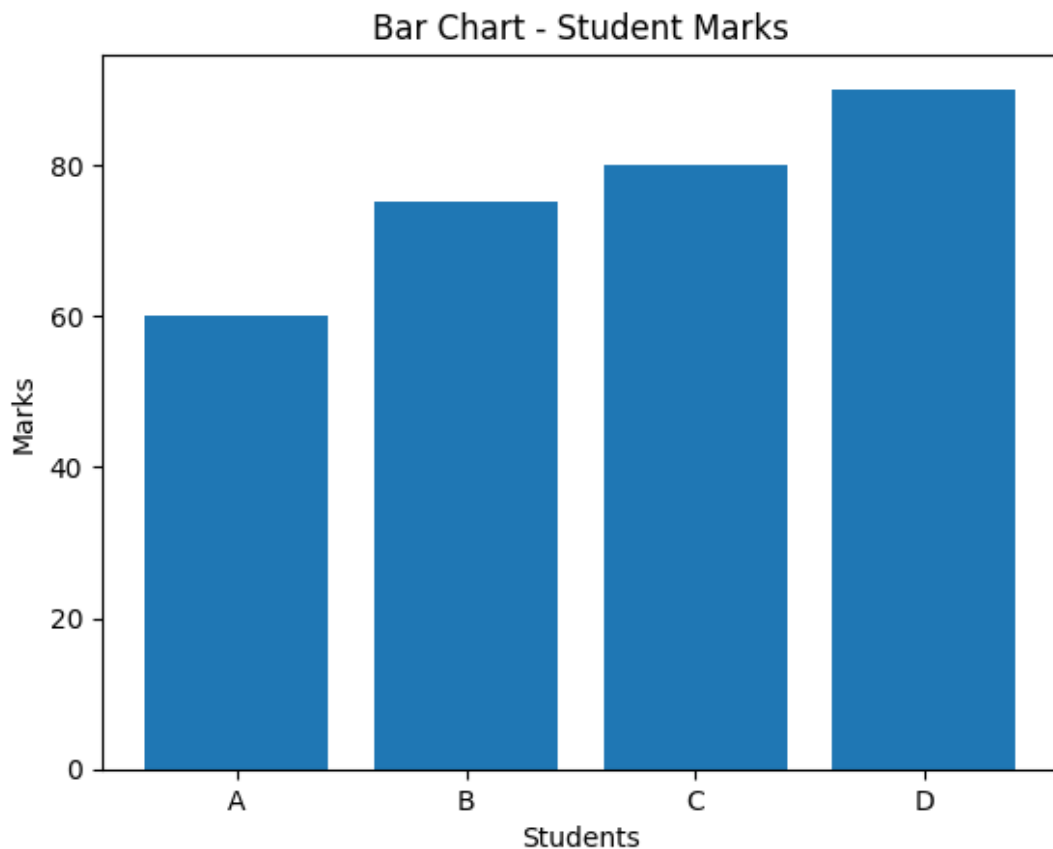
Line charts are useful when we want to observe how a variable changes over a period of time. The slope of the line indicates whether the data is increasing, decreasing, or constant.

BAR CHART

A bar chart represents data using rectangular bars, where the length or height of each bar corresponds to the value it represents.

```
#BAR CHART
import matplotlib.pyplot as plt
students = ["A", "B", "C", "D"]
marks = [60, 75, 80, 90]

plt.bar(students, marks)
plt.title("Bar Chart - Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.show()
```



Explanation:

Bar charts are ideal for comparing different categories. Each category is represented on the x-axis, while the values are shown on the y-axis.

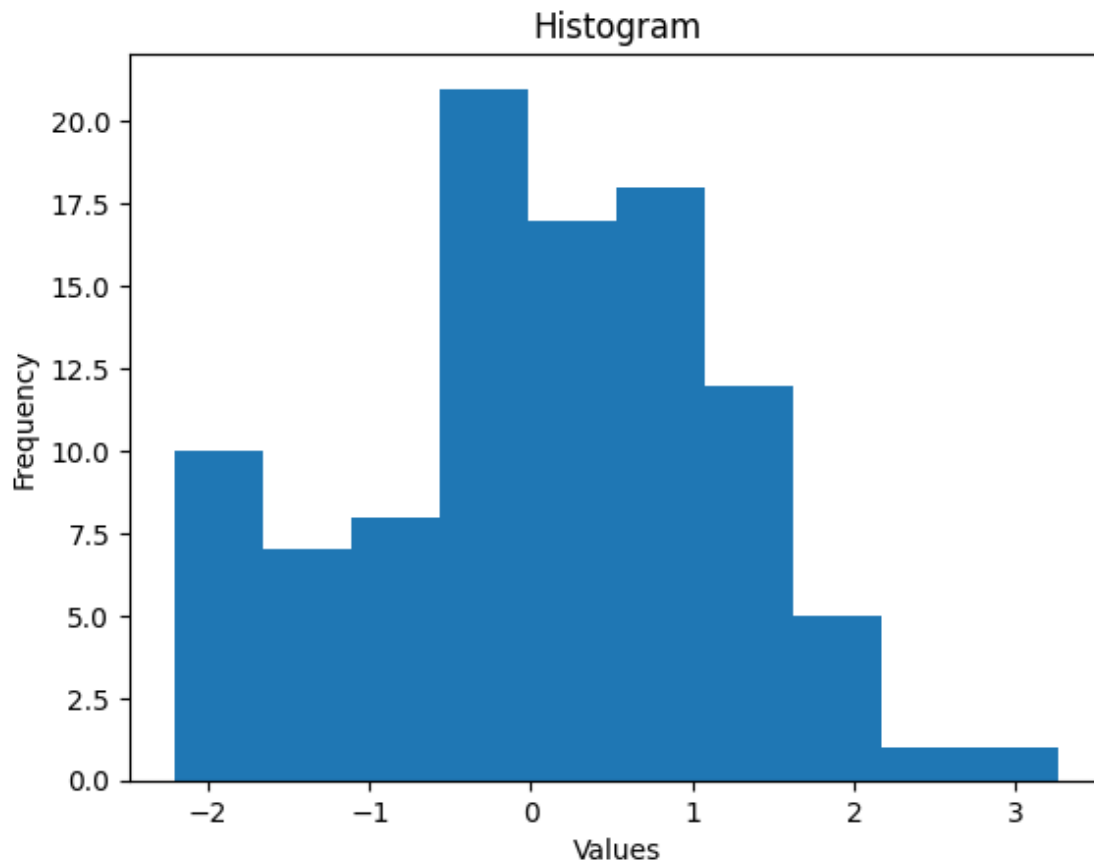
HISTOGRAM

A histogram is used to represent the distribution of numerical data. It groups data into bins or intervals and shows the frequency of values in each bin.

```
▶ #HISTOGRAM
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(100)

plt.hist(data, bins=10)
plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```



Explanation:

Histograms help in understanding the spread and shape of the data, such as whether it is normally distributed or skewed.

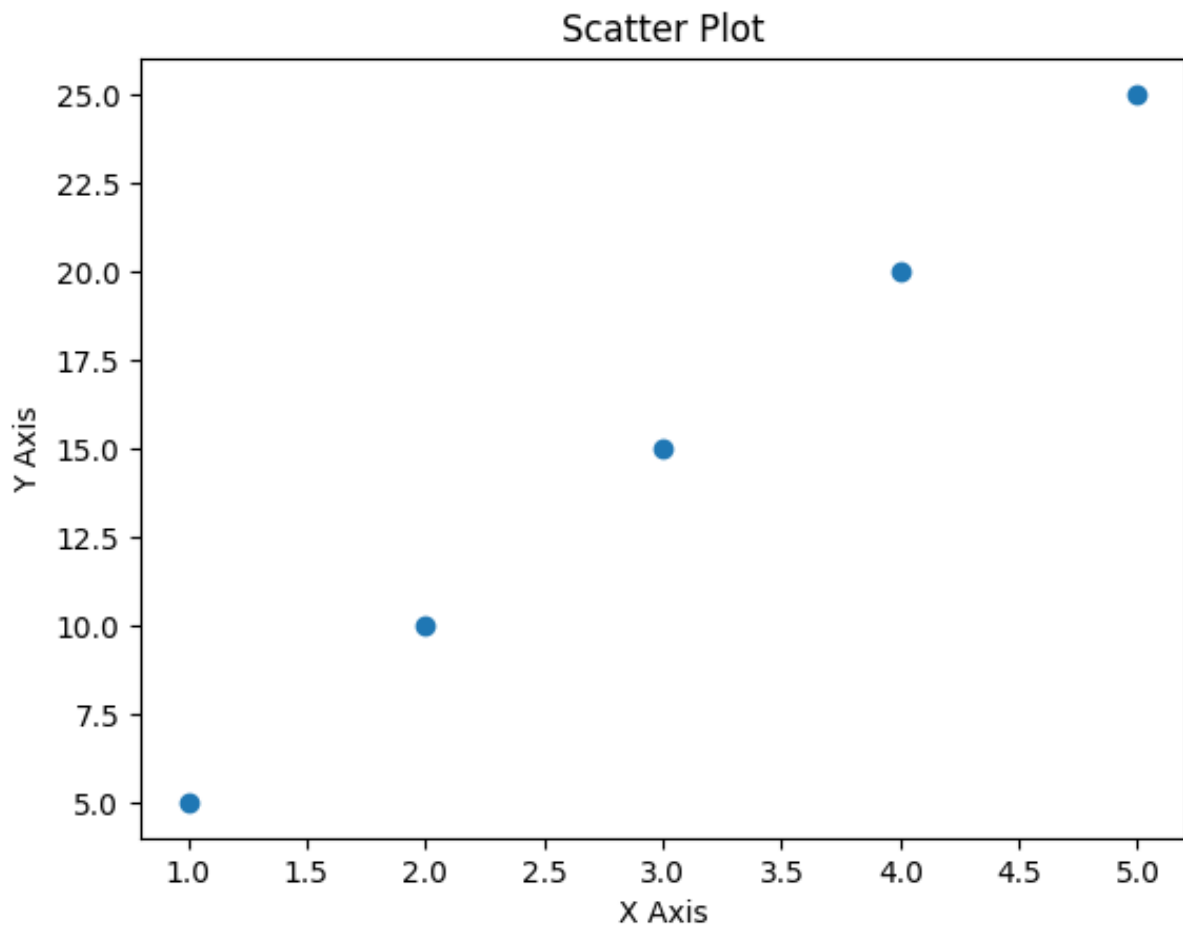
SCATTER PLOT

Scatter plots are useful for showing the relationship between two variables and identifying patterns or correlations.

```
▶ #SCATTER PLOT
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [5,10,15,20,25]

plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```



Explanation:

Pie charts are useful when we want to show proportions or percentages of a whole.

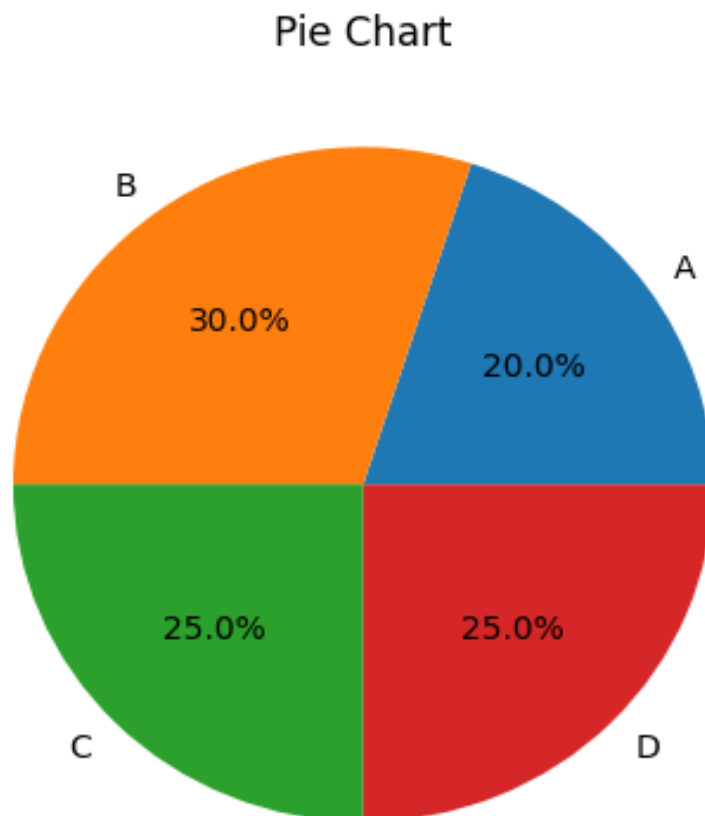
PIE CHART

A pie chart represents data as slices of a circle, where each slice corresponds to a percentage of the total.

```
#PIE CHART
import matplotlib.pyplot as plt

labels = ["A", "B", "C", "D"]
sizes = [20, 30, 25, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```



Explanation:

Pie charts are useful when we want to show proportions or percentages of a whole.

SEABORN

Seaborn is a high-level data visualization library built on top of Matplotlib. It simplifies the process of creating attractive and informative statistical graphics.

Seaborn provides built-in themes, color palettes, and functions that make it easier to create complex visualizations with less code. It also integrates well with Pandas data structures, making it highly suitable for data analysis.

Let us see the different plots in each category.

1 Regression Plots:

- `scatterplot()`
- `lineplot()`
- `relplot()`

2 Categorical Plots:

- `barplot()`
- `countplot()`
- `boxplot()`
- `violinplot()`
- `swarmplot()`
- `pointplot()`
- `catplot()`

3 Distribution Plots:

- `histplot()`
- `kdeplot()`
- `rugplot()`
- `distplot()`

4 Relational Plots:

- `regplot()`
- `lmplot()`

5 Matrix Plots:

- `heatmap()`
- `clustermap()`

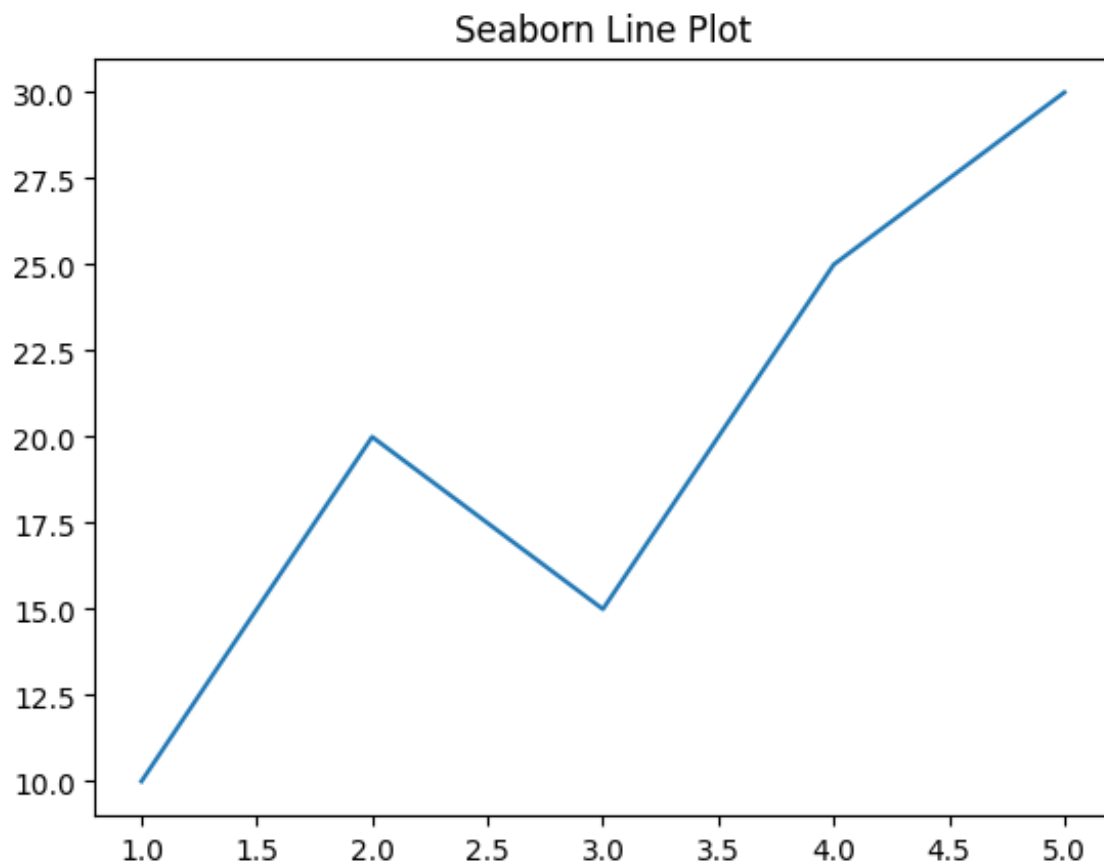
LINE PLOT

A line plot in Seaborn is used to visualize trends in data over time or ordered categories.

```
▶ #LINE PLOT
import seaborn as sns
import matplotlib.pyplot as plt

data = [10, 20, 15, 25, 30]

sns.lineplot(x=[1,2,3,4,5], y=data)
plt.title("Seaborn Line Plot")
plt.show()
```



Explanation:

It automatically handles statistical aggregation and displays confidence intervals, making it more advanced than a simple line plot in Matplotlib.

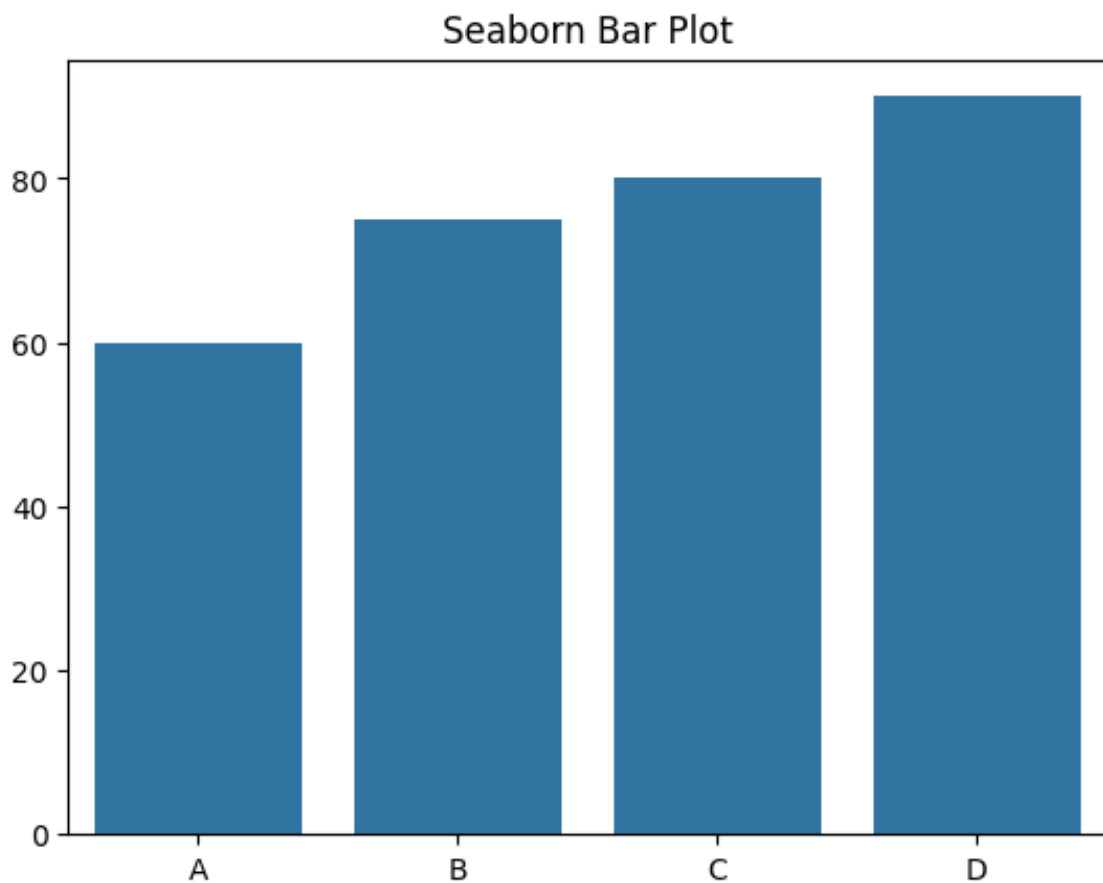
BAR PLOT

Seaborn's bar plot not only displays values but can also show statistical measures such as mean and confidence intervals.

```
▶ #BAR PLOT
import seaborn as sns
import matplotlib.pyplot as plt

students = ["A", "B", "C", "D"]
marks = [60, 75, 80, 90]

sns.barplot(x=students, y=marks)
plt.title("Seaborn Bar Plot")
plt.show()
```



Explanation:

It is commonly used to compare values across categories with better visual appearance.

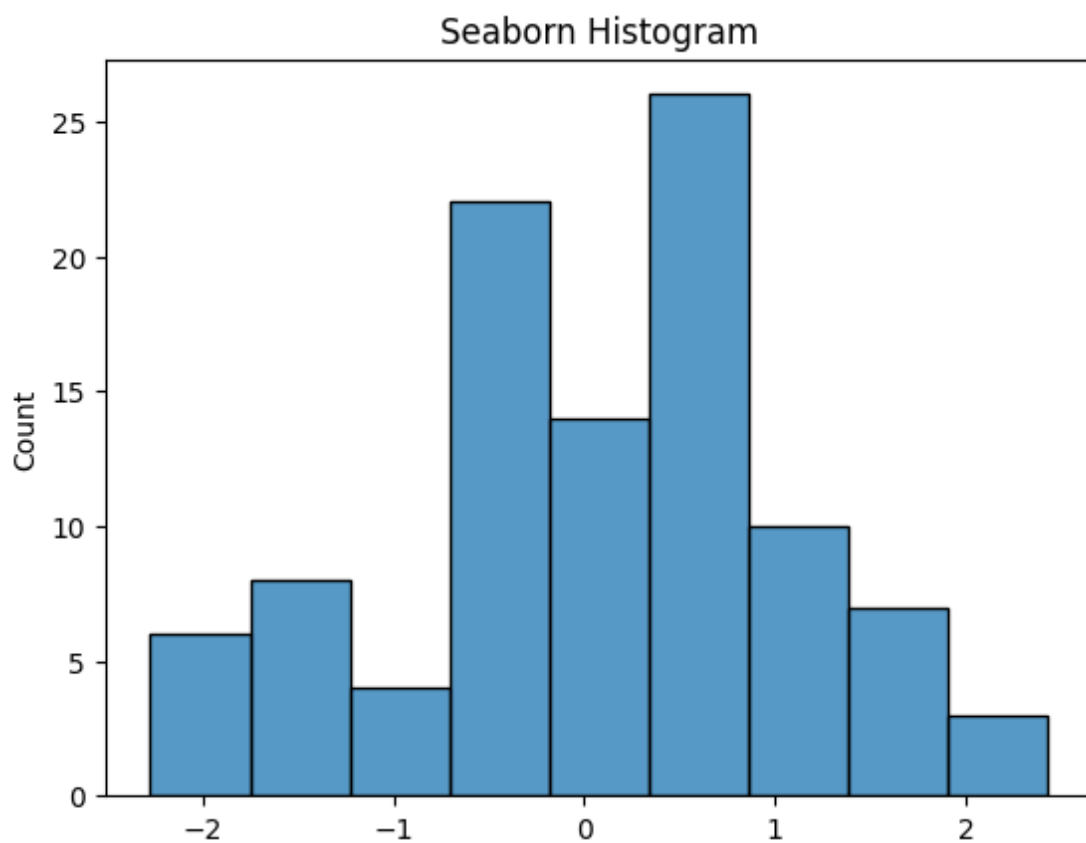
HISTOGRAM

Seaborn's histogram provides enhanced visualization of data distribution with options like KDE (Kernel Density Estimate).

```
▶ #HISTOGRAM
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(100)

sns.histplot(data)
plt.title("Seaborn Histogram")
plt.show()
```



Explanation:

It helps in understanding how data is distributed and whether it follows a normal distribution.

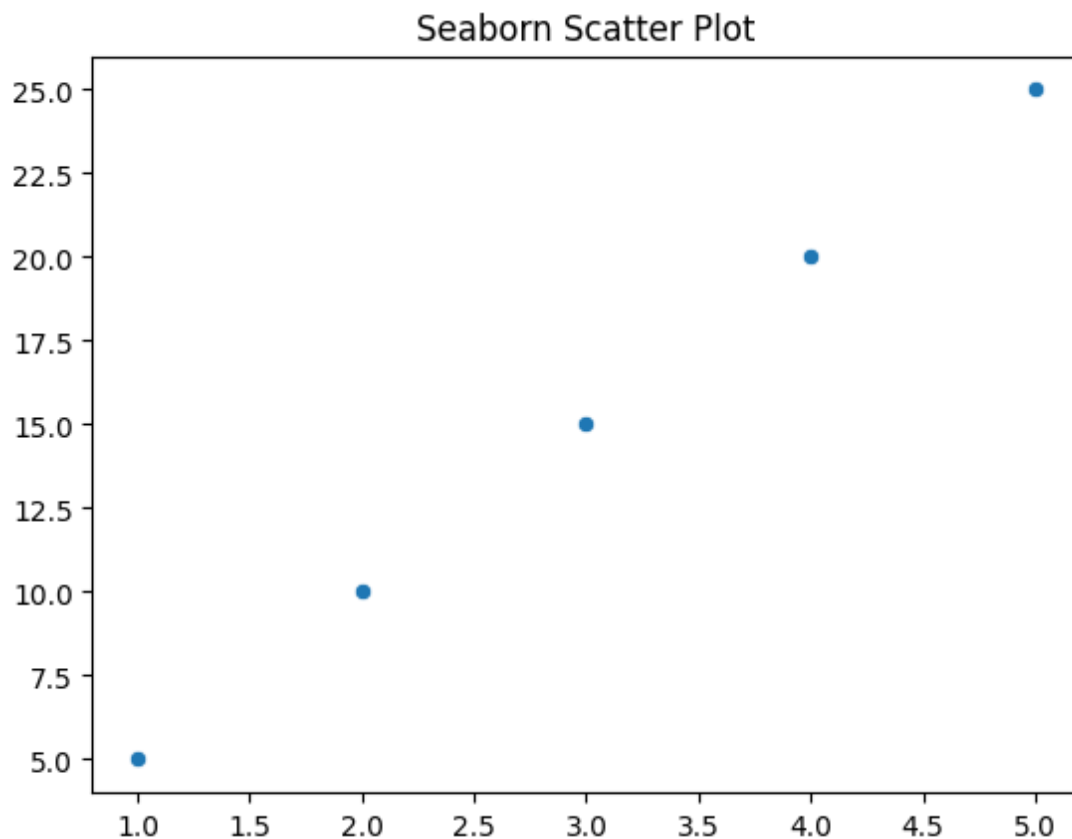
SCATTER PLOT

A scatter plot in Seaborn allows additional features such as color grouping (hue) to differentiate categories.

```
▶ #SCATTER PLOT
import seaborn as sns
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [5,10,15,20,25]

sns.scatterplot(x=x, y=y)
plt.title("Seaborn Scatter Plot")
plt.show()
```



Explanation:

This makes it easier to analyze relationships between variables across multiple groups.

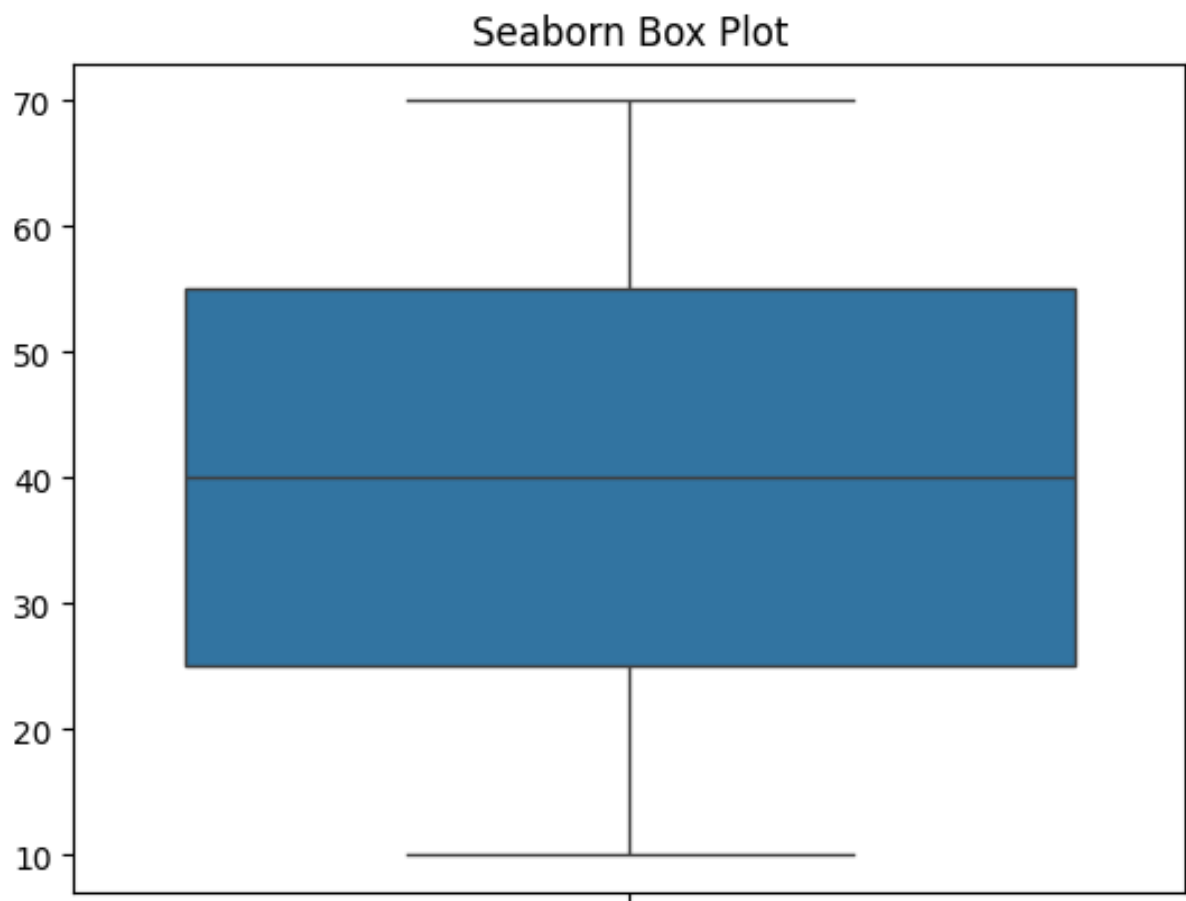
BOX PLOT

A box plot shows the distribution of data based on five summary statistics: minimum, first quartile, median, third quartile, and maximum.

```
▶ #BOX PLOT
import seaborn as sns
import matplotlib.pyplot as plt

data = [10,20,30,40,50,60,70]

sns.boxplot(data=data)
plt.title("Seaborn Box Plot")
plt.show()
```



Explanation:

Box plots are useful for understanding data spread and detecting unusual values.

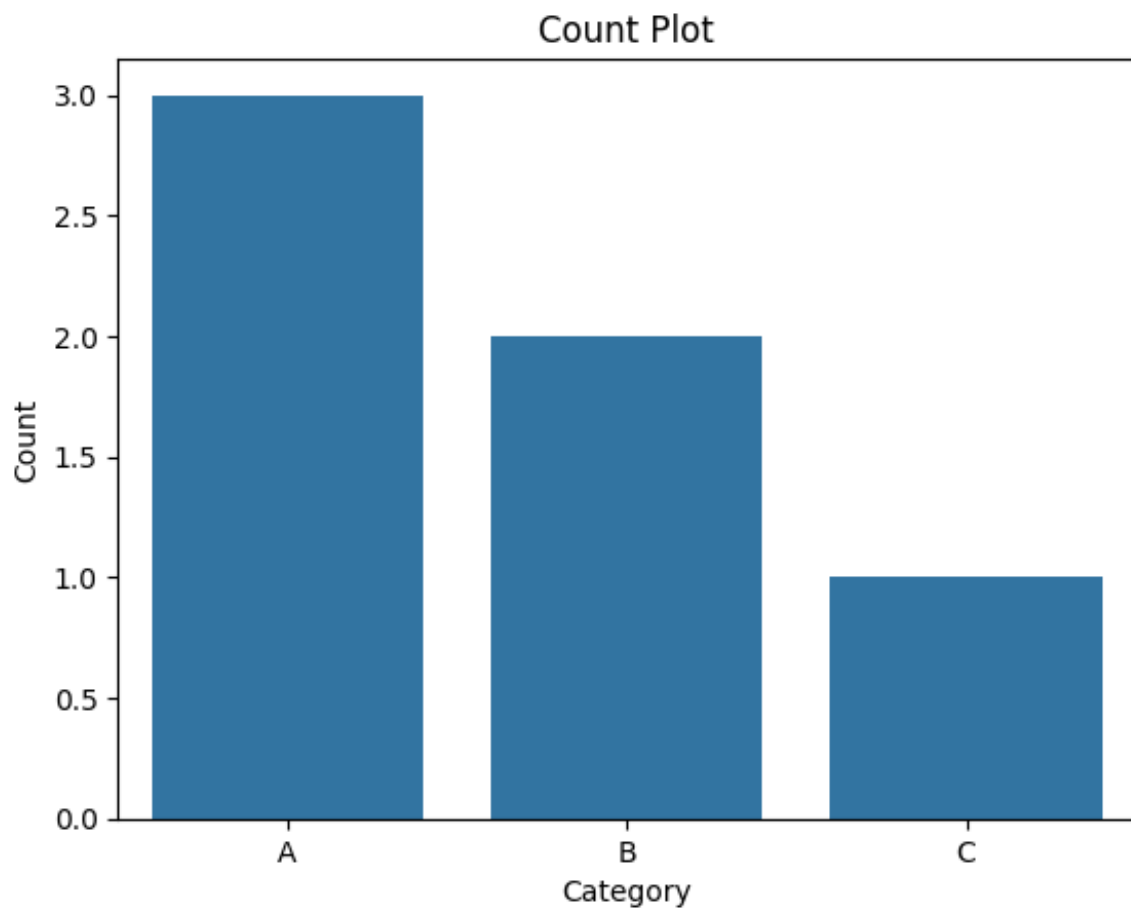
COUNT PLOT

A count plot is used to show the frequency (count) of categorical data. It counts how many times each category appears in the dataset and displays it as bars.

```
▶ import seaborn as sns
import matplotlib.pyplot as plt

data = ["A", "B", "A", "C", "B", "A"]

sns.countplot(x=data)
plt.title("Count Plot")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```



Explanation:

A count plot shows how many times each category appears in the data using bars.

COMPARISON OF MATPLOTLIB AND SEABORN

Matplotlib:

- **Core Library:** Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a low-level interface for creating plots with fine-grained control over every aspect of the figure.
- **Flexibility:** Matplotlib allows users to create any type of plot imaginable, from basic line plots and scatter plots to complex 3D plots and geographical maps.
- **Mature:** Matplotlib has been around for a long time and is a well-established library in the Python ecosystem. Many other libraries, including Seaborn, are built on top of Matplotlib.
- **Customization:** Matplotlib offers extensive customization options, allowing users to customize every aspect of a plot, including colors, line styles, markers, fonts, annotations, and more.
- **Integration:** Matplotlib can be easily integrated with other libraries and frameworks, such as NumPy, Pandas, SciPy, which makes it a versatile choice for data visualization in various contexts.

Advantages of Matplotlib:

- High degree of customization and control over plots.
- Wide range of plot types and styles.
- Strong community support and extensive documentation.
- Well-suited for creating publication-quality figures and graphics.

Seaborn:

- **High-Level Interface:** Seaborn is built on top of Matplotlib and provides a high-level interface for creating attractive and informative statistical graphics. It simplifies the process of creating complex visualizations by providing default aesthetics and convenient functions for common tasks.
- **Statistical Plotting:** Seaborn specializes in statistical plotting and offers a variety of plot types specifically designed for visualizing relationships in data, such as scatter plots, bar plots, box plots, violin plots, pair plots, and more.
- **Aesthetics:** Seaborn comes with built-in themes and color palettes that make it easy to create visually appealing plots with minimal effort. It also provides tools for fine-tuning plot aesthetics, such as controlling the size of plot elements and adjusting the color palette.
- **Integration with Pandas:** Seaborn seamlessly integrates with Pandas dataframes, allowing users to easily plot data stored in Pandas data structures without the need for extensive data manipulation.

Advantages of Seaborn:

- Simplified syntax and high-level functions for creating complex plots.
- Built-in support for statistical plotting and data exploration.
- Attractive default aesthetics and color palettes.
- Seamless integration with Pandas dataframes for data visualization.
- Ideal for exploratory data analysis and quick visualization of relationships in data.

CONCLUSION

Data visualization helps in understanding and analyzing data effectively. Both Matplotlib and Seaborn are powerful tools that allow users to create meaningful and visually appealing graphs. Learning these tools is essential for data science projects.